Hide-and-Seek Drone Swarm

CentraleSupélec Multi-Agent Systems — Technical Report

May 27, 2026

Abstract

This report describes the design and implementation of a multi-agent hide-and-seek system in which a single airborne *seeker* drone tries to spot a group of *hider* drones that hide behind obstacles. The system is built on top of the CentraleSupélec Volière TP framework: every drone controller is a Python function that the simulator (`standalone_sim.py`) calls each tick with the current world state, and must return a velocity command. The contribution of this work lies in the modules around `tp_algos.py`: an obstacle-consensus law with both an asymptotic linear term and a barrier term; a 3D field-of-view sensor; a shared-memory broadcast channel between hiders; a cover-seeking primitive that picks a point geometrically behind an obstacle relative to the last-known seeker pose; and a controller-level 3D inter-drone repulsion that lives inside `tp_algos.py` itself and adds a hard "never closer than 1 m" fence on top of the per-controller behaviours. The overall control law combines a Lissajous roaming target, a consensus-based obstacle push, an in-controller 2D inter-hider separation, a top-level 3D drone-to-drone separation, and soft boundary repulsion, with a single magnitude saturation at the actuator limit.

Contents

1 Project context and high-level architecture

The project is a CentraleSupélec TP (2A/3A) on multi-agent systems. The target hardware platforms are Crazyflie 2 nano-quadrotors (acting as hiders) and a DJI RoboMaster TT drone (acting as the seeker), flying inside the school's *Volìere* arena. The pedagogical aim is to use a hide-and-seek game as an accessible illustration of multi-agent control, both for the students implementing it and for the children visiting the Volìere.

Rules of the game.

- A single seeker drone (RMTT) cruises the arena on a slow sweep path. It *never receives the hiders' positions* – it has to find them with vision.
- Several hider drones (CF2) roam the arena. They know the seeker's position only through *their own vision* (a forward cone), and only communicate sightings to each other through a shared broadcast channel.
- When the seeker's forward cone contains a hider *and* the line of sight is not occluded by an obstacle, the hider is "caught" and triggered to land.

Module decomposition. The code base is split into small files that each address one concern:

Module	Role
`standalone_sim.py`	The simulator. Owns the world state, calls each robot's controller in a thread pool, integrates kinematics, detects collisions, renders the 3D figure. *Untouched in this project.*
`tp_algos.py`	The required entry-point file (prof requirement). Beyond the dispatch shims, it hosts a controller-level 3D inter-drone repulsion (`_compute_drone_repulsion_3d`) and applies it on top of both the seeker and hider commands. *Edits kept inside the marked `TO BE MODIFIED` blocks.*
`cf2_milestone1.py`	Top-level hider controller: composes a goal force, obstacle consensus, in-plane CF2-to-CF2 separation, and boundary repulsion; switches between roaming and cover-seeking.
`cf2_sensing.py`	The hider's sensing primitives: intent heading and FoV-cone obstacle sensing on an axis-aligned bounding box (AABB) model.
`cf2_consensus.py`	The obstacle consensus law (linear + barrier), used by both hiders and seeker.
`cf2_hider.py`	Inter-hider communication (shared seeker-sighting memory) and the cover-target geometry.
`rmtt_seeker.py`	Seeker controller (`compute_seeker_cmd`, which does *not* accept hider positions) and the `can_see` predicate shared with the referee and the hiders' seeker-detection logic.
`game_referee.py`	Per-frame catch detection using the same `can_see` predicate as the seeker's vision cone.
`sim_viz.py`	Visualization overlays: 3D FoV cones, position trails, status HUD.

Data flow. At each tick $t = k\,\Delta t$ ($\Delta t = 0.05\,\mathrm{s}$), the simulator:

1. Snapshots all pose arrays so the controllers see a consistent world.

2. Submits each robot's controller (the public `tp_algos._controller` entry points) to a thread pool. The controller returns a target velocity (and for drones, a target altitude / takeoff / land flags).
3. Integrates the returned commands with a simple Euler step, applies the actuator-rate saturations, adds a Gaussian position noise term ($\sigma = 5\,\text{mm}$ per step).
4. Calls `game_referee.update_catches` to mark any catches.
5. Runs collision detection (boundary, obstacle, robot-to-robot) and colours the robots' safety globes red on contact.
6. Calls `sim_viz.update` to refresh cones, trails and the HUD.

Default scenario. The simulator ships with `nbRMTT = 1` seeker, `nbCF2 = 1` hider (scalable up to 3 in the TP framework), and `nbObstacle = 2`: a $1.0 \times 0.5 \times 2.5\,\text{m}$ pillar at $(-0.5, 1.5, 0)$ and a $0.5 \times 1.1 \times 2.5\,\text{m}$ pillar at $(0.8, -2.0, 0)$. The two pillars sit in opposing "rooms" of the arena (positive vs. negative y), keep at least 1.4 m of clearance from every wall (well above $r_{\text{safe}} = 0.6\,\text{m}$), and are $\sim 3.7\,\text{m}$ apart center-to-center – enough that the cover-seeking hider has a real choice of which side to flee toward, while the seeker's y-dominant Lissajous sweeps both rooms on every cycle. The RMTT starts at $(-1, 1, 0)$ and is forced into takeoff at $t = 0$; the CF2 starts at $(-1, 0, 0)$ and only takes off once its controller requests it.

2 Mathematical model

2.1 State and kinematics

Each drone is modelled as a point mass with state $\mathbf{p}_i = (x_i, y_i, z_i)^\top \in \mathbb{R}^3$ and a velocity command $\mathbf{u}_i = (v_x, v_y, v_z)^\top$. The integration is a forward Euler step with Gaussian noise:

$$\mathbf{p}_i(t + \Delta t) \;=\; \mathbf{p}_i(t) + \mathbf{u}_i \Delta t + \eta_i, \qquad \eta_i \sim \mathcal{N}\big(\mathbf{0}, \sigma^2 \mathbf{I}_3\big)\,, \qquad \sigma = 0.005\,\text{m}.$$

Before integration the simulator applies a 3D magnitude clamp at the actuator limit (`clamp_vel3d` in `standalone_sim.py`).

For CF2 the controller actually returns $(v_x, v_y, z_{\text{dist}})$ where z_{dist} is the *desired altitude*. The simulator converts it to a vertical velocity $v_z = z_{\text{dist}} - z$ and applies the 3D clamp on (v_x, v_y, v_z) together.

2.2 Obstacles

Obstacles are axis-aligned bounding boxes (AABBs) standing on the floor:

$$\mathcal{B}_j \;=\; [o_x - \tfrac{s_x}{2},\, o_x + \tfrac{s_x}{2}] \times [o_y - \tfrac{s_y}{2},\, o_y + \tfrac{s_y}{2}] \times [0,\, s_z].$$

The closest point on $\mathcal{B}_j$ to a query point $\mathbf{p}$ is the per-axis clamp of $\mathbf{p}$ into the box, which collapses to $\mathbf{p}$ itself when $\mathbf{p} \in \mathcal{B}_j$.

3 Hider controller (`cf2_milestone1.py`)

3.1 Force composition

The controller assembles four terms:

$$\mathbf{F}_{\text{total}} \;=\; \mathbf{F}_{\text{goal}} + \mathbf{F}_{\text{obs}} + \mathbf{F}_{\text{sep}} + \mathbf{F}_{\text{bnd}},$$

and outputs the planar component of $\mathbf{F}_{\text{total}}$ after a magnitude saturation at $v_{\max} = 0.45\,\text{m/s}$ (chosen below the hardware cap MAX_V_CF2 $= 0.6\,\text{m/s}$ to leave headroom for the vertical channel inside the simulator's 3D clamp).

Goal attraction

$$\mathbf{F}_{\text{goal}} \;=\; k_g\,(\mathbf{p}_{\text{goal}} - \mathbf{p}), \qquad k_g = 0.65.$$

The goal is either the drone's Lissajous waypoint (Sec. 3.2) or its cover target (Sec. 3.4), depending on whether the swarm is in roaming or hide mode.

Obstacle consensus

Each sensed obstacle j contributes (see Sec. 4)

$$\mathbf{F}^{(j)}_{\text{obs}} \;=\; \underbrace{k_{\text{obs}}\,\max(0, r_{\text{safe}} - d_j)}_{\text{linear}}\,\mathbf{n}_j \;+\; \underbrace{k_{\text{bar}}\left(\frac{r_{\text{safe}}}{\max(d_j, \varepsilon)} - 1\right)}_{\text{barrier}}\,\mathbf{n}_j,$$

with d_j the distance to the obstacle surface, $\mathbf{n}_j$ the outward normal (surface $\to$ drone), and $\varepsilon = 0.02\,r_{\text{safe}}$.

In-plane CF2-to-CF2 separation

For each other hider k within $d_0 = 0.9\,\text{m}$:

$$\mathbf{F}^{(k)}_{\text{sep}} \;=\; g_{\text{sep}}\,\frac{1}{d_{ik}^2}\left(\frac{1}{d_{ik}} - \frac{1}{d_0}\right)\,\hat{\mathbf{e}}_{ki}, \qquad g_{\text{sep}} = 5.0,$$

with $\hat{\mathbf{e}}_{ki} = (\mathbf{p}_i - \mathbf{p}_k)/\|\mathbf{p}_i - \mathbf{p}_k\|$ restricted to the xy plane. The shape vanishes smoothly at $d = d_0$ and grows unbounded as $d \to 0$ – the same philosophy as the obstacle barrier, but between drones. This is a soft, planar inter-hider repulsion; a stricter 3D "never closer than 1 m" separation between *all* flying drones (hiders *and* seeker) is added on top in `tp_algos.py` (Sec. 10).

Boundary repulsion

Each wall at distance d inside a margin $m = 0.6\,\text{m}$ pushes inward:

$$F^{(w)}_{\text{bnd}} \;=\; g_{\text{bnd}}\left(\frac{1}{\max(d, 10^{-4})} - \frac{1}{m}\right), \qquad g_{\text{bnd}} = 0.09.$$

The gain is intentionally small (0.09) – the goal is a gentle nudge, not a hard bounce.

3.2 Lissajous roaming path

Each hider i tracks a 3D Lissajous waypoint

$$\mathbf{p}^{(i)}_{\text{goal}}(t) \;=\; \begin{pmatrix} c_x + A_x \sin(\omega_x^{(i)} t + \varphi_x^{(i)}) \\ c_y + A_y \sin(\omega_y^{(i)} t + \varphi_y^{(i)}) \\ c_z \end{pmatrix},$$

with $A_x = 1.6$, $A_y = 3.0$, $c_z = 1.0$, and per-axis, per-drone angular frequencies

$$\omega_x^{(i)} = 0.16 + 0.018\,(i-1), \quad \omega_y^{(i)} = 0.11 + 0.015\,(i-1), \quad \omega_z^{(i)} = 0.09 + 0.013\,(i-1)\ \text{rad/s},$$

and coprime phase offsets $\varphi_x^{(i)} = (i-1)\cdot 2\pi/3$, $\varphi_y^{(i)} = (i-1)\cdot 2\pi/5$, $\varphi_z^{(i)} = (i-1)\cdot 2\pi/7$. The three axes use different base frequencies (not a single $0.16 + 0.018(i-1)$) so a single drone alone already traces a non-degenerate Lissajous, and the small per-drone increments stop the swarm from collapsing onto a shared orbit.

Two intentionally different "Lissajous targets". There are *two* versions of the Lissajous formula in the code base, and they differ on purpose:

- `cf2_milestone1._lissajous_target` (used as the position goal for $\mathtt{F}_{\text{goal}}$) returns $t_z = c_z$, i.e. the $A_z \sin(\cdot)$ term is commented out so the drone holds a constant cruise altitude and the on-screen cone stays at a predictable height.
- `cf2_sensing.intent_heading` (used to build the FoV-cone *axis*) returns $t_z = c_z + A_z \sin(\omega_z^{(i)} t + \varphi_z^{(i)})$, $A_z = 0.5$, so the cone gets a gentle vertical sweep – handy if the seeker is at a slightly different altitude.

Position tracking is altitude-locked, but *sensing* is not, which keeps detection ranges 3D without forcing the controller to chase z.

3.3 Mode switching: roam vs. hide

The hiders communicate seeker sightings through a single module-level dict in `cf2_hider.py`. The mode logic in `compute_roaming_cmd` is:

1. Compute the FoV-based seeker detection: if `can_see(self, seeker, intent_heading, FOV_HALF_ANGLE = 60°, range = 3.0 m)`, call `report_seeker_sighting(seeker_pos, t)`.
2. Read the shared memory via `get_last_seen(t)`. If a sighting is fresher than TTL = 8 s, switch the goal target to the cover point (next section); otherwise, use the Lissajous target.

The structural asymmetry is important: the seeker controller never reads or writes the shared memory, so "hiders can talk to each other but the seeker can't listen" is enforced by interface, not by convention.

3.4 Cover-seeking geometry

Given a last-known seeker position $\mathbf{s}$, the function `seek_cover_target` picks an xy point behind one of the obstacles. For each obstacle with center $\mathbf{o}_j$ and half-extents (h_x, h_y):

1. Form the unit direction $\mathbf{u} = (\mathbf{o}_j - \mathbf{s})/\|\mathbf{o}_j - \mathbf{s}\|$ from the seeker to the obstacle center.
2. Compute the parametric exit distance of the ray $\mathbf{o}_j + t\,\mathbf{u}$ from the obstacle's AABB:

$$t_{\text{surf}} \;=\; \min\!\left(\frac{h_x}{|u_x|}, \frac{h_y}{|u_y|}\right),$$

   with the obvious degenerate handling when one of u_x, u_y is zero.
3. The cover candidate is $\mathbf{c}_j = \mathbf{o}_j + (t_{\text{surf}} + d_{\text{cov}})\,\mathbf{u}$, with $d_{\text{cov}} = 0.8\,\text{m}$.

The hider picks the candidate closest to its own position. The $d_{\text{cov}} > r_{\text{safe}}$ inequality is deliberate: the cover point sits *outside* the obstacle-consensus push zone, so the hider can come to rest on the cover point without fighting the avoidance law.

A pure-flee fallback (move 1 m further from the seeker along the hider-to-seeker direction) is used when no obstacle is usable – e.g. when the seeker happens to stand at an obstacle center.

4 Obstacle consensus law (`cf2_consensus.py`)

4.1 Definition

Given a list of sensed obstacles, each with closest-surface distance d_j and outward normal $\mathbf{n}_j$, the consensus command is

$$\mathbf{u} \;=\; \sum_{j\,:\,d_j<r_{\text{safe}}} \left[k_{\text{obs}}\,(r_{\text{safe}}-d_j) + k_{\text{bar}}\left(\frac{r_{\text{safe}}}{\max(d_j,\varepsilon)}-1\right)\right]\mathbf{n}_j.$$

The function does *not* saturate; saturation is the caller's responsibility (the hider controller applies the magnitude cap on the summed force, after adding the goal/separation/boundary contributions).

4.2 Properties

Asymptotic (linear term). Define $e_j(\mathbf{p}) = \max(0, r_{\text{safe}} - d_j(\mathbf{p}))$ and the Lyapunov candidate

$$V(\mathbf{p}) \;=\; \tfrac{1}{2}\sum_j e_j(\mathbf{p})^2.$$

For static obstacles, $\nabla d_j = \mathbf{n}_j$ on the violating set, so the purely-linear closed loop $\dot{\mathbf{p}} = -k_{\text{obs}}\sum_j e_j\mathbf{n}_j$ satisfies $\dot{V} = -k_{\text{obs}}\sum_j e_j^2 \le 0$, with equality only when all violations vanish. The drone therefore converges to the safe set $\{\mathbf{p} : d_j(\mathbf{p}) \ge r_{\text{safe}}\ \forall j\}$ – an asymptotic consensus on "be outside every safety bubble".

Bounded actuation (barrier term). The linear term alone has bounded magnitude on bounded violations, so a strong enough goal-attraction force could overpower it close to the obstacle. The barrier term grows like r_{safe}/d_j as $d_j \to 0$, which *always* dominates any bounded attraction once the drone is close enough. Combined with the actuator saturation in the caller, the closed-loop velocity is bounded everywhere while the dynamics inside $\{d_j < r_{\text{safe}}\}$ remain dominated by the avoidance push.

Numerical safety. The denominator uses $\max(d_j, \varepsilon)$ with $\varepsilon = 0.02\,r_{\text{safe}} = 12\,\text{mm}$. In simulation a drone may briefly penetrate an obstacle due to the discrete integration step; this clamp keeps the law finite while still producing a very large push.

5 Sensing (`cf2_sensing.py`)

5.1 Intent heading

The heading used both for the FoV cone and the sensing axis is the unit vector toward the current Lissajous target:

$$\hat{\mathbf{h}}(t) \;=\; \frac{\mathbf{p}_{\text{goal}}(t) - \mathbf{p}(t)}{\|\mathbf{p}_{\text{goal}}(t) - \mathbf{p}(t)\|}.$$

Compared to a path-tangent heading $\dot{\mathbf{p}}_{\text{goal}}/\|\dot{\mathbf{p}}_{\text{goal}}\|$, this points where the drone wants to be rather than where the path is currently moving. The practical consequence: when an obstacle is between the drone and its waypoint, the cone is already aimed at the obstacle.

5.2 FoV-cone obstacle sensing

For each obstacle $\mathcal{B}_j$, the function `sense_obstacles`:

1. Computes the closest box point $\mathbf{c}_j$ to the drone (per-axis clamp).
2. Computes the 3D distance $d_j = \|\mathbf{c}_j - \mathbf{p}\|$ and rejects the obstacle if $d_j >$ `sensing_radius`.
3. Applies the angular gate $\hat{\mathbf{h}} \cdot \hat{\mathbf{e}}_j \geq \cos\theta_{\text{fov}}$ where $\hat{\mathbf{e}}_j = (\mathbf{c}_j - \mathbf{p})/d_j$, returning the obstacle's $\mathbf{n}_j = -\hat{\mathbf{e}}_j$ and d_j.

A degenerate case is handled explicitly: if $d_j < 10^{-6}$ (drone is on or inside the obstacle) the function flags it as in-cone with $\mathbf{n}_j = -\hat{\mathbf{h}}$, so the downstream consensus law pushes the drone back along the inverse of its heading.

Why hiders sense omnidirectionally. The hider controller passes $\theta_{\text{fov}} = \pi$ to `sense_obstacles`, effectively disabling the angular gate. The forward 60° cone is reserved for *seeker detection*, not obstacle avoidance, since otherwise a hider would be free to enter an obstacle from a blind side.

6 Seeker controller (`rmtt_seeker.py`)

6.1 Sweep path

The seeker tracks a slow Lissajous sweep:

$$\mathbf{p}^S_{\text{goal}}(t) = \begin{pmatrix} A^S_X \sin(\Omega_X t) \\ A^S_Y \sin(\Omega_Y t + \pi/2) \\ c^S_z + A^S_Z \sin(\Omega_Z t) \end{pmatrix},$$

with $A^S_X = 1.8$, $A^S_Y = 3.5$, $A^S_Z = 0.4$, $c^S_z = 1.0$, and angular frequencies $\Omega_X = 0.18$, $\Omega_Y = 0.13$, $\Omega_Z = 0.08$ rad/s (cycle times $\approx 35, 48, 79$ s). The $\pi/2$ phase between x and y produces a figure-of-eight footprint that sweeps both "rooms" of the arena every cycle.

6.2 Control law

The seeker uses the same consensus law as the hiders (with the same parameters $r_{\text{safe}} = 0.6$, $k_{\text{obs}} = 2$, $k_{\text{bar}} = 0.5$) plus a goal attraction $\mathbf{F}^S_{\text{goal}} = k^S_g(\mathbf{p}^S_{\text{goal}} - \mathbf{p})$, $k^S_g = 0.6$. Saturation is applied separately on the planar magnitude (at $v^S_{\max,xy} = 0.55$ m/s) and on the vertical component (at $|v^S_z| \leq 0.35$ m/s).

The signature `compute_seeker_cmd(robot_no, robot_pose, obstacle_pose, obstacle_size, clock)` deliberately does *not* accept hider positions: the seeker's *navigation* cannot use any hider state. The `tp_algos.rmtt_control_fn` wrapper does receive `cf2_poses` (its signature is fixed by the TP framework) and uses them *only* to add the 3D safety-fence repulsion described in Sec. 10 – it never forwards them to `compute_seeker_cmd`, so the controller's planning logic remains hider-blind.

6.3 Search cone and line-of-sight

The search cone has half-angle $\theta_{\text{seek}} = 30°$ and range 3.5 m – narrower but longer than the hiders' cone. The predicate `can_see(apex, target, heading, fov_half, range, obstacles)` returns `True` when:

1. The 3D distance $\|\mathbf{t} - \mathbf{a}\| \leq$ range.
2. The cone condition holds: $(\mathbf{t} - \mathbf{a}) \cdot \hat{\mathbf{h}} \geq \cos\theta_{\text{fov}} \|\mathbf{t} - \mathbf{a}\|$.

3. No obstacle box intersects the line segment from apex to target.

The occlusion test uses the standard Kay–Kajiya slab algorithm (`_segment_intersects_aabb`): for each of the 3 axes, compute the parameter interval $[t_1, t_2]$ over which the segment lies between the slab faces; intersect the three intervals; the segment hits the box iff the intersection overlaps $[0, 1]$. Axis-parallel segments are handled with an explicit "inside the slab?" check.

7 Inter-hider communication (`cf2_hider.py`)

The shared state is a single module-level dict:

```
_last_seen = {'position': None, 'timestamp': None}
SEEKER_MEMORY_TTL = 8.0  # seconds
```

Writers (`report_seeker_sighting`) overwrite both fields atomically; readers (`get_last_seen`) compare `t - timestamp` against the TTL and return either the position or `None`. Last-write-wins is fine here because the question is "what is the most recent fix on the seeker", and between two sightings the older one offers no extra information.

TTL choice. 8 s is comfortably longer than the time the seeker takes to cross the arena (roughly half of its 48 s y-cycle, so ≈ 24 s end-to-end) but much shorter than a full sweep. Practically: the swarm stays in hide mode while the threat is plausibly nearby, and returns to roaming within $\sim$8 s after the seeker has moved out of sight.

Threading. The simulator runs each controller in a `ThreadPoolExecutor`. Python's GIL plus the atomic-dict-write convention is enough here: a reader can never see a half-written record, only a fully old or fully new one.

8 Catch detection (`game_referee.py`)

The referee owns a `set` of caught hider IDs and a dict of catch times. Each frame it iterates over (flying seekers) $\times$ (flying, not-yet-caught hiders), calls `can_see` (the same predicate the seeker would compute internally if it had one), and adds the hider to the caught set if visible. Once caught, a hider stays caught for the rest of the simulation. The hider controller in `tp_algos.cf2_control_fn` reads this flag via `is_caught(robot_no)` and returns `trigger_land = True`, which puts the drone into landing state.

9 Visualization (`sim_viz.py`)

The visualization layer adds three things on top of the simulator's basic markers:

- **3D FoV cones.** For each flying drone, a triangle-fan polyhedron is built with apex at the drone and axis along its heading. The 16-segment base disk is constructed in the plane perpendicular to the heading using an orthonormal basis built from an arbitrary reference vector (world-z, switching to world-x when the heading is nearly vertical).
- **Position trails.** Each hider keeps a bounded deque (`TRAIL_LEN = 80`) of recent positions, drawn as a thin green line. When the hider returns to idle (after a catch landing) the trail is cleared so the next takeoff starts fresh.
- **HUD.** A small monospace text in the upper left shows t, the catch count, and an `alert on/off` flag derived from `get_last_seen(t)`.

The cone colour for the hiders switches from `limegreen` (roam) to `gold` (alert) when a fresh sighting is in shared memory, so the global mode is visible at a glance.

10 Simulator hooks and controller-level separation (`tp_algos.py`)

The TP framework requires every controller to live in `tp_algos.py` with a fixed signature. Per the project rules, only the body marked `TO BE MODIFIED` may be edited. Two responsibilities live here:

1. Dispatch each robot type to its dedicated module (`cf2_milestone1`, `rmtt_seeker`, ...).
2. Apply a controller-level 3D drone-to-drone separation *on top of* what each module returns. This is the only place in the code base that has simultaneous access to *both* hider and seeker poses, so it is the natural place to enforce a global "no two drones get closer than 1 m" rule.

10.1 3D inter-drone repulsion

The helper `_compute_drone_repulsion_3d(self_pose, neighbors, min_dist=1.0)` computes, for each neighbor closer than $d_{\min}$, a unit push along the 3D vector from neighbor to self, scaled by a linear deficit:

$$\Delta \mathbf{v}_i \;=\; g_{\text{rep}} \sum_{k\,:\,d_{ik}<d_{\min}} \frac{d_{\min}-d_{ik}}{d_{\min}} \, \frac{\mathbf{p}_i - \mathbf{p}_k}{\|\mathbf{p}_i - \mathbf{p}_k\|}, \qquad g_{\text{rep}} = 1.0,\; d_{\min} = 1.0\,\text{m}.$$

A degenerate case ($d_{ik} < \varepsilon = 10^{-6}$ m) returns a fixed $+x$ push so the function never divides by zero. The sum operates in all three axes, contrary to the planar separation in `cf2_milestone1`, which matters when a hider and the seeker happen to be vertically aligned.

10.2 How the helper is used

- `cf2_control_fn`: once the hider is flying and is not caught, it calls `compute_roaming_cmd` for $(v_x, v_y, z_{\text{dist}}, \text{led})$. It then collects *all other* flying CF2s *and all* RMTT seekers as neighbors, asks `_compute_drone_repulsion_3d` for $(\Delta v_x, \Delta v_y, \Delta v_z)$, and adds the result to the controller's (v_x, v_y) and to a synthesized $v_z = z_{\text{dist}} - z$. The corrected v_z is then folded back into z_{dist} so the simulator's "$v_z = z_{\text{dist}} - z$" convention is preserved.
- `rmtt_control_fn`: gets $(v_x, v_y, v_z, \text{led})$ from `compute_seeker_cmd` (which itself never sees hider poses), then collects all flying CF2s as neighbors and adds the 3D repulsion to (v_x, v_y, v_z). This is the one place where the seeker controller layer actually reads `cf2_poses` – but only to keep its distance, not to chase. The asymmetry of the game (the seeker has to *see* the hiders to catch them) is still preserved because the catch logic in `game_referee` uses vision alone.
- `tb3B_control_fn`, `tb3W_control_fn`, `rmep_control_fn`: placeholder goal-seek behaviour ($v_x, v_y = 0.2(g - p)$ toward a hard-coded g). These robots are not part of the current hide-and-seek configuration (`nbTb3B` = `nbTb3W` = `nbRMEP` = 0).

10.3 Hider takeoff handshake

The CF2 takeoff is split between the simulator's state machine and a small per-controller cache in `cf2_control_fn`. The cache (`_takeoff_done_by_robot`, indexed by `robot_no`) starts at `False`; while the drone sits at $z < 0.05$ m it returns `trigger_takeoff = True` and shows a blue LED. The simulator then runs the 3 s linear takeoff (state 1) up to `CF2_HOVER_Z`; once the controller sees $z > 0.1$ m it flips the cache to `True` and from then on delegates to `compute_roaming_cmd`. If the referee marks the hider caught, the wrapper sends `trigger_land = True` and lights the LED red.

The lazy-import trick (`if not hasattr(..., "_cached"): import ...`) keeps the module fast to reload and avoids polluting the framework's expected namespace.

11 Simulation infrastructure (`standalone_sim.py`)

11.1 World and hardware specs

Constant	Value	Meaning
`dt`	0.05 s	Simulation step.
`X_MIN, X_MAX`	± 2.5 m	Arena bounds (mirrored in `cf2_milestone1`).
`Y_MIN, Y_MAX`	± 4.5 m	
`Z_MIN, Z_MAX`	0, 3.5 m	
`MAX_V_CF2`	0.6 m/s	3D speed cap for hider drones.
`MAX_V_RMTT`	0.6 m/s	3D speed cap for the seeker.
`RAD_CF2`	0.05 m	CF2 safety glob is $2\times$ this.
`RAD_RMTT`	0.08 m	RMTT safety glob is $2\times$ this.
`CF2_HOVER_Z`	1.0 m	Target hover altitude after takeoff.
`RMTT_HOVER_Z`	0.8 m	
`TAKEOFF_TIME`	3.0 s	Linear takeoff over $[0, \text{HOVER\_Z}]$.
`LANDING_TIME`	3.0 s	
`DRONE_POS_NOISE_STD`	0.005 m	Per-step Gaussian noise on each axis.

11.2 State machine

Each drone has a 4-state machine:

$$0\ \text{Idle} \xrightarrow{\text{trigger\_takeoff}} 1\ \text{Takeoff} \xrightarrow{\text{timer expired}} 2\ \text{Flying} \xrightarrow{\text{trigger\_land}} 3\ \text{Landing} \xrightarrow{\text{timer/altitude}} 0.$$

In states 1 and 3 the altitude is driven linearly by the takeoff/landing timer; in state 2 the controller's (v_x, v_y, v_z) are applied with the actuator saturation and noise step.

11.3 Async controller execution

The simulator runs each controller in a `ThreadPoolExecutor` with `max_workers = 20`. The helper `get_async_cmd` submits a job if none is running for the given robot, and otherwise returns the cached last command. This means a controller can block (e.g. `time.sleep`) without stalling the simulation – the drone simply keeps its last command. The trade-off is that controllers can be running on slightly stale snapshots, but in practice the world barely changes in one `dt`.

11.4 Collision detection

Each robot has a "safety glob" of radius $2 \times$ base that turns red on contact. Three checks per frame:

- **Boundary**: arena walls in x, y; ceiling at $z = Z_{\max}$ always; floor at $z = 0$ only for drones in the flying state.
- **Obstacle**: closest-point distance against each AABB (same per-axis-clamp formula as the sensing module).
- **Robot-to-robot**: 3D distance against the sum of the two globs' radii.

Warnings are rate-limited to one per (robot, kind) per second.

12 Tunables summary

Name	Value	Where / why
r_{safe}	0.6 m	Desired clearance from obstacles (`R_SAFE` in both hider and seeker).
k_{obs}	2 (1/s)	Linear consensus gain.
k_{bar}	0.5 m/s	Barrier gain near contact.
ε	$0.02\, r_{\text{safe}}$	Numerical clamp inside the barrier.
θ_{fov} hiders	60°	Forward cone for seeker detection and visualization.
sensing radius	2.0 m	Obstacle sensing range (hider and seeker).
seeker detection range	3.0 m	Hider-side range to detect the seeker.
d_0 (sep)	0.9 m	In-plane CF2-to-CF2 separation influence radius (`cf2_milestone1`).
g_{sep}	5	In-plane CF2-to-CF2 separation gain.
$d_{\min}$	1.0 m	3D drone-to-drone hard-fence trigger (`tp_algos.DRONE_MIN_DISTANCE`).
g_{rep}	1.0	3D drone-to-drone repulsion gain (`tp_algos.DRONE_REPULSION_GAIN`).
boundary margin	0.6 m	Distance at which wall push turns on.
g_{bnd}	0.09	Wall push gain.
$v_{\max}^{\text{xy}}$ hider	0.45 m/s	Final xy saturation, < MAX_V_CF2.
Z rate limit	0.3 m	Per-call $\lvert z_{\text{dist}} - z \rvert$ cap.
d_{cov}	0.8 m	Cover offset past the obstacle surface.
TTL	8 s	Lifetime of a seeker sighting in shared memory.
θ_{seek}	30°	Seeker FoV half-angle.
seeker vision range	3.5 m	Seeker FoV slant length.
$\Omega_X^S, \Omega_Y^S, \Omega_Z^S$	0.18, 0.13, 0.08 rad/s	Seeker Lissajous.
A_X^S, A_Y^S, A_Z^S	1.8, 3.5, 0.4 m	Seeker Lissajous.
$\omega_x^{(i)}$	$0.16 + 0.018(i-1)$ rad/s	Hider Lissajous (per-drone, x).
$\omega_y^{(i)}$	$0.11 + 0.015(i-1)$ rad/s	Hider Lissajous (per-drone, y).
$\omega_z^{(i)}$	$0.09 + 0.013(i-1)$ rad/s	Hider Lissajous (per-drone, z; used in `intent_heading` only).
A_x, A_y	1.6, 3.0 m	Hider Lissajous amplitudes in xy.
A_z	0.5 m	Used only in `intent_heading` (FoV axis z); disabled in the position target.
TRAIL_LEN	80 frames	Trail length in the visualization.

13 Discussion and design choices

Why separate the FoV cone from the obstacle sensor. The first iteration of the hider used the same 60° cone for both seeker detection and obstacle avoidance. The result: the

drone would happily back into a wall when its heading was pointing forward. Splitting the two (omnidirectional for obstacles, forward cone for vision) is what makes the hider safe in a cluttered arena while still "looking somewhere".

Why a barrier on top of the linear consensus. A pure linear law with the same k_{obs} would let the goal-attraction push the drone arbitrarily close to an obstacle if the goal is on the far side. The barrier term grows like r_{safe}/d, so it eventually dominates any *bounded* attraction; combined with the magnitude saturation at the actuator limit, this gives "always-bounded velocity, never collides" under reasonable goals.

Why cover behind the surface, not the center. For elongated obstacles, a cover point at distance r_{cov} from the obstacle center can still be inside the AABB. The implementation computes the parametric ray-AABB exit and adds the offset *past* the surface; combined with $d_{\text{cov}} > r_{\text{safe}}$ this gives a cover point that sits in the safe set, so the hider can rest there without being pushed away.

Why hide the hider positions from the seeker controller specifically. The *planning* module of the seeker, `compute_seeker_cmd`, does not accept `cf2_poses`. The `tp_algos` wrapper still has access to them (its signature is dictated by the framework) and uses them only for the 3D safety-fence repulsion. The split is deliberate: a safety fence is a property of the world, not of the game (we do not want drones colliding mid-air); whereas the search behaviour must remain vision-only, which is what the referee enforces via `can_see`. Future changes to the seeker's search logic would have to explicitly thread hider positions through `compute_seeker_cmd`'s signature, which would be visible in review.

Why shared-memory comms rather than message passing. The simulator already runs each controller in a thread; a single last-write-wins dict is the smallest faithful model of "every hider can hear every other hider's report". A more elaborate scheme (per-pair channels, ack/nak, etc.) would not change the closed-loop behaviour because the controllers only care about "most recent fix on the seeker".

14 Future work

- **Multiple seekers and adversarial cooperation.** The `can_see` predicate is already multi-seeker aware (the referee loops over all flying RMTTs). What is missing is a seeker coordination law that splits the arena and avoids redundant sweeps.
- **Bounded/asymptotic consensus toward the swarm centroid.** The current inter-hider law is repulsive only. Adding a weak attractive term toward the swarm centroid would produce the bounded-consensus behaviour mentioned in the project brief.
- **Per-hider altitude diversity.** The A_z term is already threaded through `intent_heading` but stripped from `_lissajous_target`, so the FoV cone wobbles in z while the position is altitude-locked. Re-enabling the A_z component in the position target would spread the hiders vertically, making the seeker's narrow cone less effective; the Z rate limit (0.3 m/call) is already small enough that this would not destabilize the actuator saturation.
- **Persistent cover assignment.** The current `seek_cover_target` picks the nearest cover per drone every tick; under heavy cover demand two hiders may oscillate on the same side of the same obstacle. A simple "don't reconsider for τ seconds" rule would prevent the chatter.

A File listing

File	LoC (approx.)	Purpose
`cf2_milestone1.py`	~ 200	Main hider controller.
`cf2_sensing.py`	~ 150	FoV sensing primitives.
`cf2_consensus.py`	~ 60	Obstacle consensus law.
`cf2_hider.py`	~ 130	Inter-hider comms + cover geometry.
`rmtt_seeker.py`	~ 200	Seeker controller + `can_see`.
`game_referee.py`	~ 60	Catch detection.
`sim_viz.py`	~ 170	Cones, trails, HUD.
`tp_algos.py`	~ 360	Required entry-point shim plus controller-level 3D drone-to-drone repulsion.
`standalone_sim.py`	~ 420	The simulator (provided).